



DEI

DEPARTAMENTO
DE ENGENHARIA INFORMÁTICA

TÉCNICO LISBOA

Sieve of Eratosthenes

All-Pairs Shortest Paths

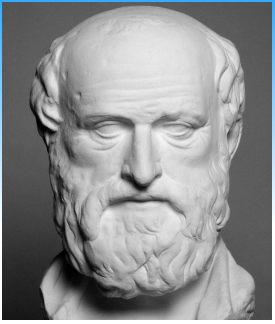
Parallel and Distributed Computing

José Monteiro

Monday, 17 March 2025

Outline

- Application examples with MPI
 - The Sieve of Eratosthenes
 - All-Pairs Shortest Paths, Floyd-Warshall's Algorithm



The Sieve of Eratosthenes

PT: Peneira



- Algorithm for finding all primes up to n , proposed by Greek mathematician Eratosthenes (276-194 BC)

1. Create list of unmarked natural numbers $2, 3, \dots, n$
2. $k = 2$
3. Repeat
 - i) Mark all multiples of k between k^2 and n
 - ii) $k =$ smallest unmarked number $> k$until $k^2 > n$
4. The unmarked numbers are primes

Complexity: $\Theta(n \log \log n)$

Example with $n=60$

1. Create list of unmarked natural numbers 2, 3, ..., n
2. $k = 2$
3. Repeat
 - i) Mark all multiples of k between k^2 and n
 - ii) $k =$ smallest unmarked number $> k$until $k^2 > n$
4. The unmarked numbers are primes

	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60

Foster's Methodology on the Sieve of Eratosthenes

- **Partitioning**
 - Make the testing of each element of the list a primitive task
- **Communication**
 - Need to send the sieve value, k , to all tasks
- **Agglomeration + Mapping**
 - What's the best way to partition the list?
 - Interleaved (cyclic) data decomposition
 - Easy to determine “owner” of each index
 - Leads to load imbalance for this problem
 - Block data decomposition
 - Balances loads
 - More complicated to determine owner if n not a multiple of p

Block Decomposition

“The devil is in the details”

- What's the best way to distribute n elements over p tasks?

Some tasks get $\left\lceil \frac{n}{p} \right\rceil$ elements, others get $\left\lfloor \frac{n}{p} \right\rfloor$.

- Which task gets which size?
- **One approach:** group larger blocks at lower index tasks
 - Let $r = n \bmod p$
 - If $r = 0$, assign $\frac{n}{p}$ elements per task
 - Otherwise,
 - First r tasks get $\left\lceil \frac{n}{p} \right\rceil$
 - Remaining tasks get $\left\lfloor \frac{n}{p} \right\rfloor$

Block Decomposition

- Range of elements that each task gets?

- First element of task i : $i \left\lfloor \frac{n}{p} \right\rfloor + \min(i, r)$

- Last element of task i : $(i + 1) \left\lfloor \frac{n}{p} \right\rfloor + \min(i + 1, r) - 1$

- Task owner of element j : $\max \left(\left\lfloor \frac{j}{\left\lfloor \frac{n}{p} \right\rfloor} \right\rfloor, \left\lfloor \frac{j - r}{\left\lfloor \frac{n}{p} \right\rfloor} \right\rfloor \right)$

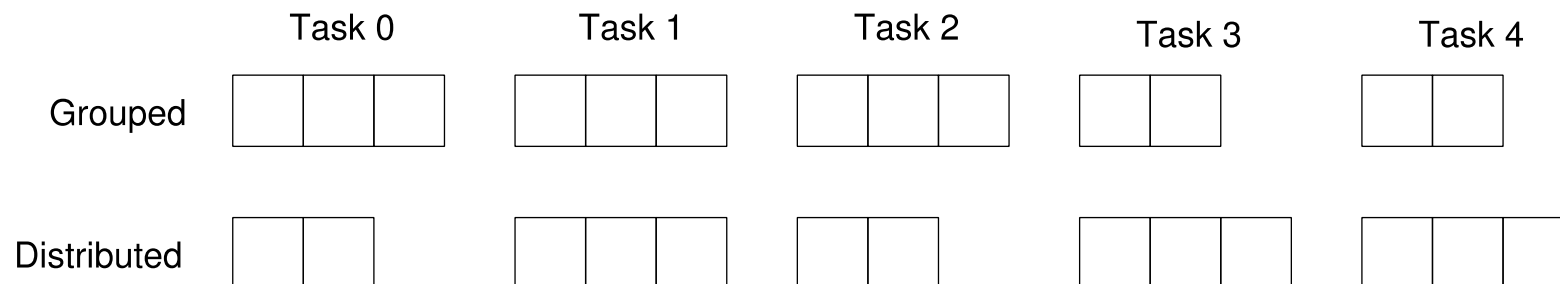
Block Decomposition

- **Grouped approach:** group larger blocks at lower index tasks

- First element of task i : $i \left\lfloor \frac{n}{p} \right\rfloor + \min(i, r)$
- Last element of task i : $(i + 1) \left\lfloor \frac{n}{p} \right\rfloor + \min(i + 1, r) - 1$
- Task owner of element j : $\max \left(\left\lfloor \frac{j}{\left\lfloor \frac{n}{p} \right\rfloor} \right\rfloor, \left\lfloor \frac{j - r}{\left\lfloor \frac{n}{p} \right\rfloor} \right\rfloor \right)$

- **Distributed approach:** distribute larger blocks evenly

- First element of task i : $i \left\lfloor \frac{n}{p} \right\rfloor$
- Last element of task i : $(i + 1) \left\lfloor \frac{n}{p} \right\rfloor - 1$
- Task owner of element j : $\left\lfloor \frac{p(j + 1) - 1}{n} \right\rfloor$



Block Decomposition Macros

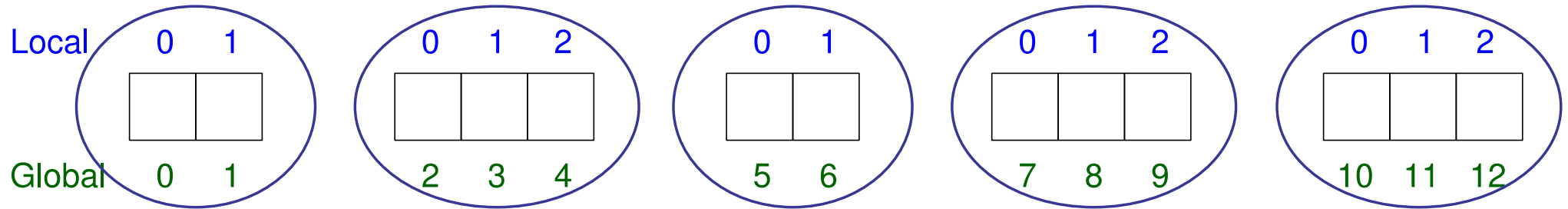
```
#define BLOCK_LOW(id,p,n) ((id)*(n)/(p))
```

```
#define BLOCK_HIGH(id,p,n) (BLOCK_LOW((id)+1,p,n) - 1)
```

```
#define BLOCK_SIZE(id,p,n) \
    (BLOCK_HIGH(id,p,n) - BLOCK_LOW(id,p,n) + 1)
```

```
#define BLOCK_OWNER(index,p,n) \
    (((p)*((index)+1)-1)/(n))
```

Local vs. Global Indexes



- Sequential program:

```
for (i = 0; i < n; i++) {  
    ...  
}
```

- Parallel program

```
size = BLOCK_SIZE(id, p, n);  
for (i = 0; i < size; i++) {  
    gi = i + BLOCK_LOW(id, p, n);  
}
```

Parallel Implementation

1. Create list of unmarked natural numbers $2, 3, \dots, n$
2. $k = 2$
3. Repeat
 - i) Mark all multiples of k between k^2 and n
 - ii) $k =$ smallest unmarked number $> k$until $k^2 > n$
4. The unmarked numbers are primes

- Use distributed block decomposition
- First task responsible for selecting next sieving prime
 - Broadcast new sieving prime at end of each iteration
 - First task has first $\lfloor n/p \rfloor$ elements, make sure it includes the largest prime used to sieve, $\sqrt{n}$
- Our program will return the **number** of primes up to n

True for $n > p^2$

Parallel Implementation

1. Create list of unmarked natural numbers 2, 3, ..., n

Each process creates its share of the list

2. $k = 2$

All processes perform this

3. Repeat

i) Mark all multiples of k between k^2 and n

Each process marks its share of the list

ii) $k =$ smallest unmarked number $> k$

Process 0 only and broadcasts it

until $k^2 > n$

4. The unmarked numbers are primes

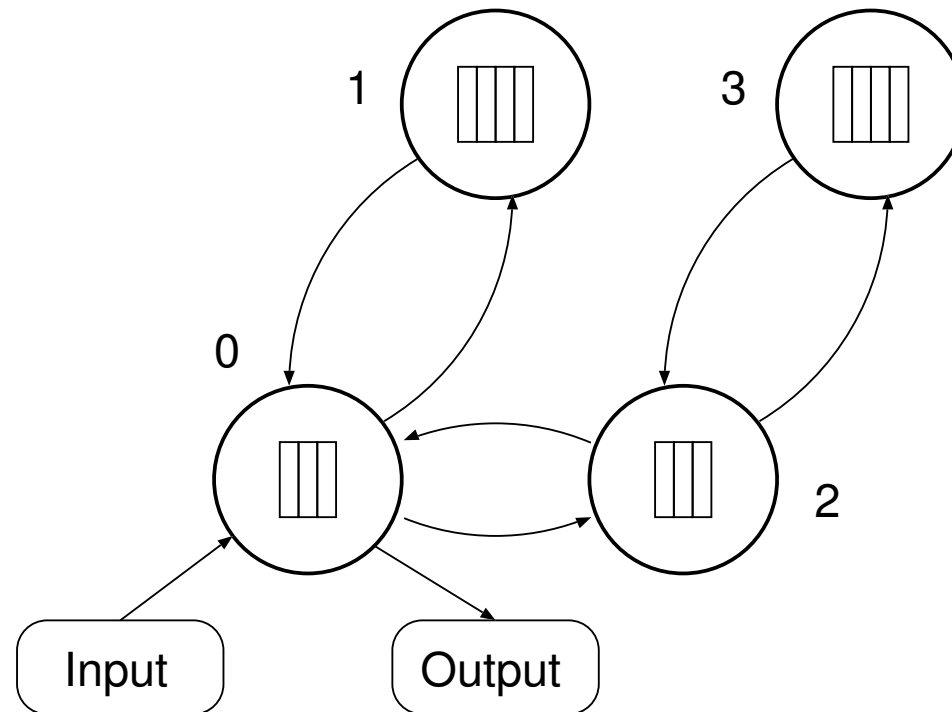
Reduction to compute the number of primes

Function MPI_Bcast

```
int MPI_Bcast (  
    void *buffer,          /* Addr of 1st element */  
    int count,             /* # elements to broadcast */  
    MPI_Datatype datatype, /* Type of elements */  
    int root,              /* ID of root process */  
    MPI_Comm comm)        /* Communicator */
```

```
MPI_Bcast (&k, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

Task / Channel Graph



Tasks organized in a **Binomial Tree** for broadcast and reduction operations.

Analysis of the Parallel Algorithm

- Let α be the time to mark a cell
- Sequential execution time: $\alpha n \log \log n$
- Computation time of parallel program: $\alpha n \log \log n / p$
- Number of broadcasts: $\sqrt{n} / \log \sqrt{n}$
- Broadcast time: $\lambda \lceil \log p \rceil$
- Reduction time: $\lambda \lceil \log p \rceil$

Prime number
theorem

- Expected parallel execution time:

$$\alpha \frac{n \log \log n}{p} + \lambda \frac{\sqrt{n} \lceil \log p \rceil}{\log \sqrt{n}} + \lambda \lceil \log p \rceil$$

Code (1/4)

```
#include <mpi.h>
#include <math.h>
#include <stdio.h>
#define BLOCK_LOW(id,p,n) ((id)*(n)/(p))
#define BLOCK_HIGH(id,p,n) (BLOCK_LOW((id)+1,p,n)-1)
#define BLOCK_SIZE(id,p,n) (BLOCK_HIGH(id,p,n)-BLOCK_LOW(id,p,n)+1)

int main (int argc, char *argv[])
{
    ...
    MPI_Init (&argc, &argv);
    MPI_Barrier(MPI_COMM_WORLD);
    elapsed_time = -MPI_Wtime();
    MPI_Comm_rank (MPI_COMM_WORLD, &id);
    MPI_Comm_size (MPI_COMM_WORLD, &p);
    if (argc != 2) {
        if (!id) printf ("Command line: %s <m>\n", argv[0]);
        MPI_Finalize(); exit (1);
    }
}
```


Code (2/4)

```
n = atoi(argv[1]);
low_value = 2 + BLOCK_LOW(id,p,n-1);
high_value = 2 + BLOCK_HIGH(id,p,n-1);
size = BLOCK_SIZE(id,p,n-1);
proc0_size = (n-1)/p;
if ((2 + proc0_size) < (int) sqrt((double) n)) {
    if (!id) printf ("Too many processes\n");
    MPI_Finalize(); exit (1);
}

marked = (char *) malloc (size);
if (marked == NULL) {
    printf ("Cannot allocate enough memory\n");
    MPI_Finalize();
    exit (1);
}

for (i = 0; i < size; i++) marked[i] = 0;
```

Code (3/4)

```
if (!id) index = 0;
prime = 2;
do {
    if (prime * prime > low_value)
        first = prime * prime - low_value;
    else {
        if (!(low_value % prime))
            first = 0;
        else
            first = prime - (low_value % prime);
    }
    for (i = first; i < size; i += prime) marked[i] = 1;
    if (!id) {
        while (marked[++index]);
        prime = index + 2;
    }
    MPI_Bcast (&prime, 1, MPI_INT, 0, MPI_COMM_WORLD);
} while (prime * prime <= n);
```

Code (4/4)

```
count = 0;
for (i = 0; i < size; i++)
    if (!marked[i]) count++;

MPI_Reduce (&count, &global_count, 1, MPI_INT, MPI_SUM,
            0, MPI_COMM_WORLD);

elapsed_time += MPI_Wtime();
if (!id) {
    printf ("%d primes are less than or equal to %d\n",
            global_count, n);
    printf ("Total elapsed time: %10.6f\n", elapsed_time);
}

MPI_Finalize ();
return 0;
}
```

Benchmarking

- Expected parallel execution time

$$\alpha \frac{n \log \log n}{p} + \lambda \frac{\sqrt{n} \lceil \log p \rceil}{\log \sqrt{n}} + \lambda \lceil \log p \rceil$$

- Experimental estimation of α , with $n = 10^8$, runtime = 24,9s

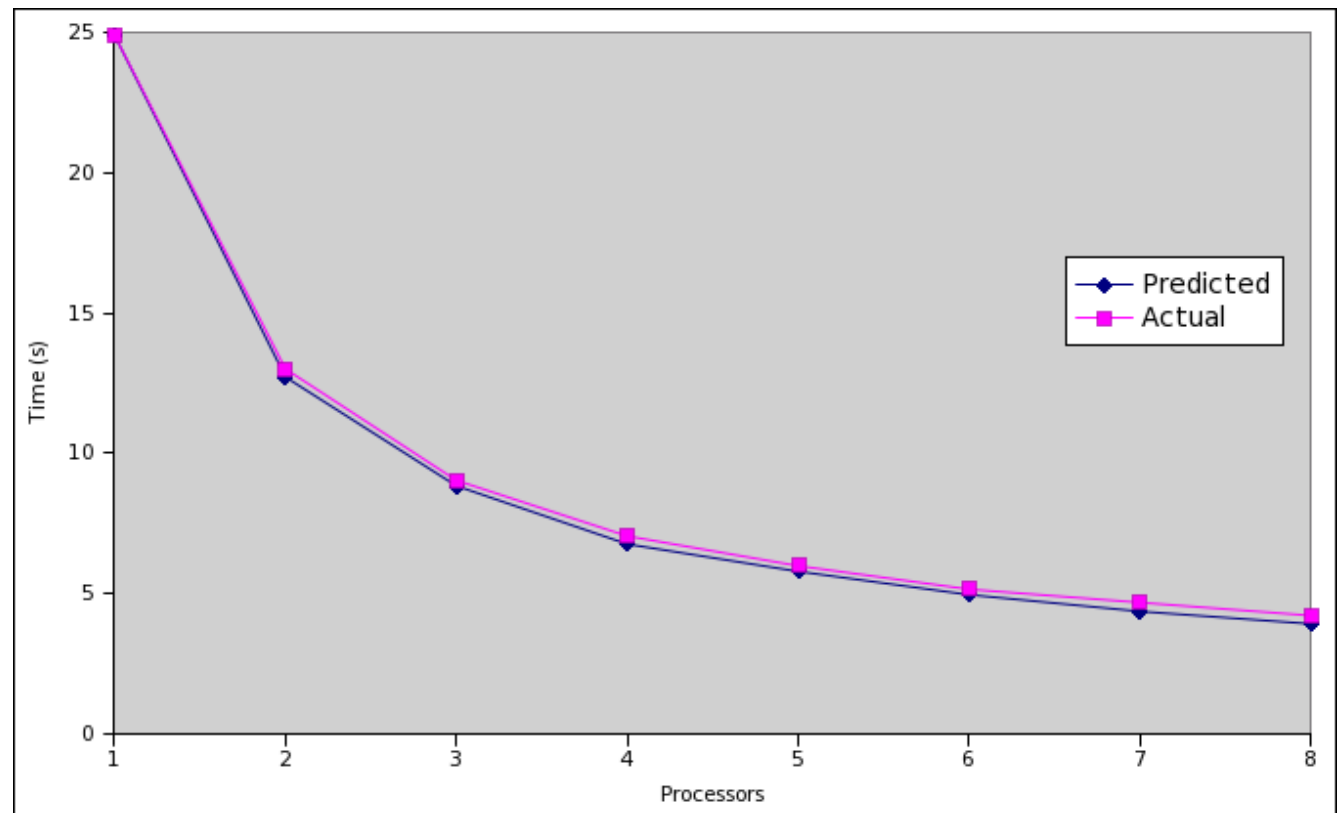
$$\alpha = \frac{24,9}{10^8 \log \log 10^8} = 52,64 \text{ ns}$$

- Experimental estimation of λ ,
with $n = 10^8$, runtimes were measured and compared
against the above formula, for $p = 2, 3, \dots, 8$ processors.

$$\lambda = 250 \mu\text{s}$$

Experimental Results

p	Predicted	Actual
1	24.9	24.9
2	12.7	13.0
3	8.8	9.0
4	6.8	7.1
5	5.8	6.0
6	5.0	5.2
7	4.4	4.7
8	3.9	4.2

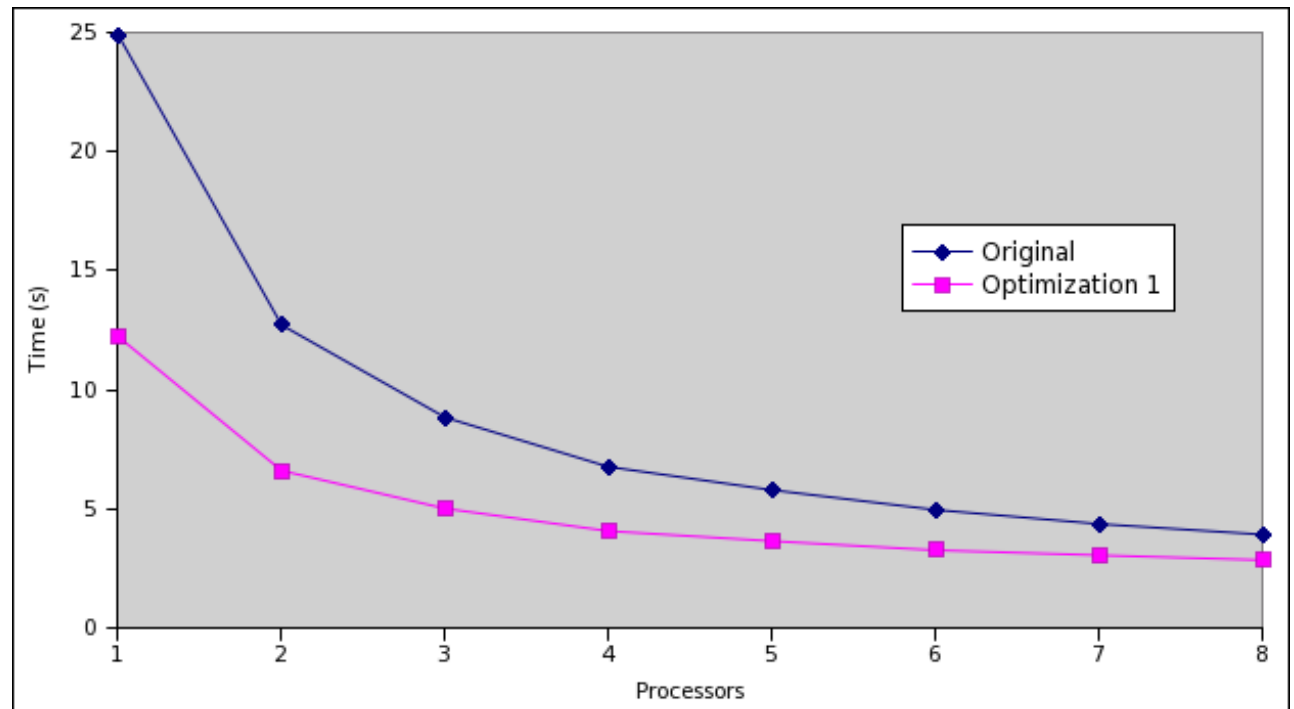


Improvements to the Program I

- Every other mark is on an even number → delete them!
 - New expected parallel execution time:

$$\alpha \frac{n \log \log n}{2p} + \lambda \frac{\sqrt{n} \lceil \log p \rceil}{\log \sqrt{n}} + \lambda \lceil \log p \rceil$$

p	Original	Optimized
1	24.9	12.2
2	13.0	6.6
3	9.0	5.0
4	7.1	4.1
5	6.9	3.7
6	5.2	3.3
7	4.7	3.1
8	4.2	2.9



Improvements to the Program II

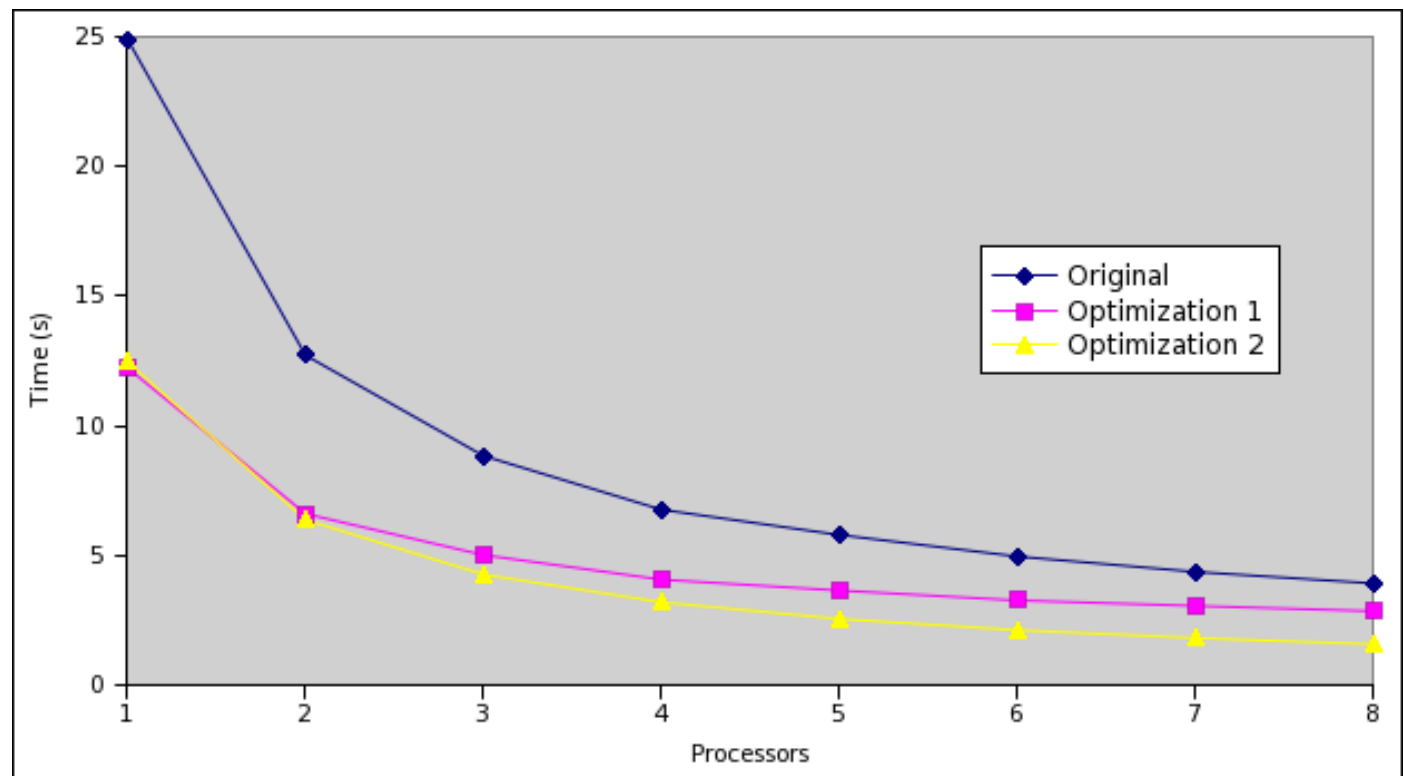
- Minimize communication
 - Duplicate sieve computation on all tasks to avoid broadcast
 - Exchange communication time by computation time
 - Replace $\lambda \frac{\sqrt{n} \lceil \log p \rceil}{\log \sqrt{n}}$ by $\alpha \sqrt{n} \log \log \sqrt{n}$
- New expected parallel execution time

$$\alpha \left(\frac{n \log \log n}{2p} + \sqrt{n} \log \log \sqrt{n} \right) + \lambda \lceil \log p \rceil$$

Improvements to the Program II

- Execution times after this second improvement

p	Original	Optimized
1	24.9	12.5
2	13.0	6.4
3	9.0	4.3
4	7.1	3.2
5	6.9	2.6
6	5.2	2.1
7	4.7	1.8
8	4.2	1.6



Improvements to the Program III

- Memory access pattern?

3	5	7	9
11	13	15	17
19	21	23	25
27	29	31	33
35	37	39	41
43	45	47	49
51	53	55	57
59	61	63	65
67	69	71	73
75	77	79	81

19	21	23	25
35	37	39	41
43	45	47	49
51	53	55	57
59	61	63	65
75	77	79	81

43	45	47	49
59	61	63	65
75	77	79	81

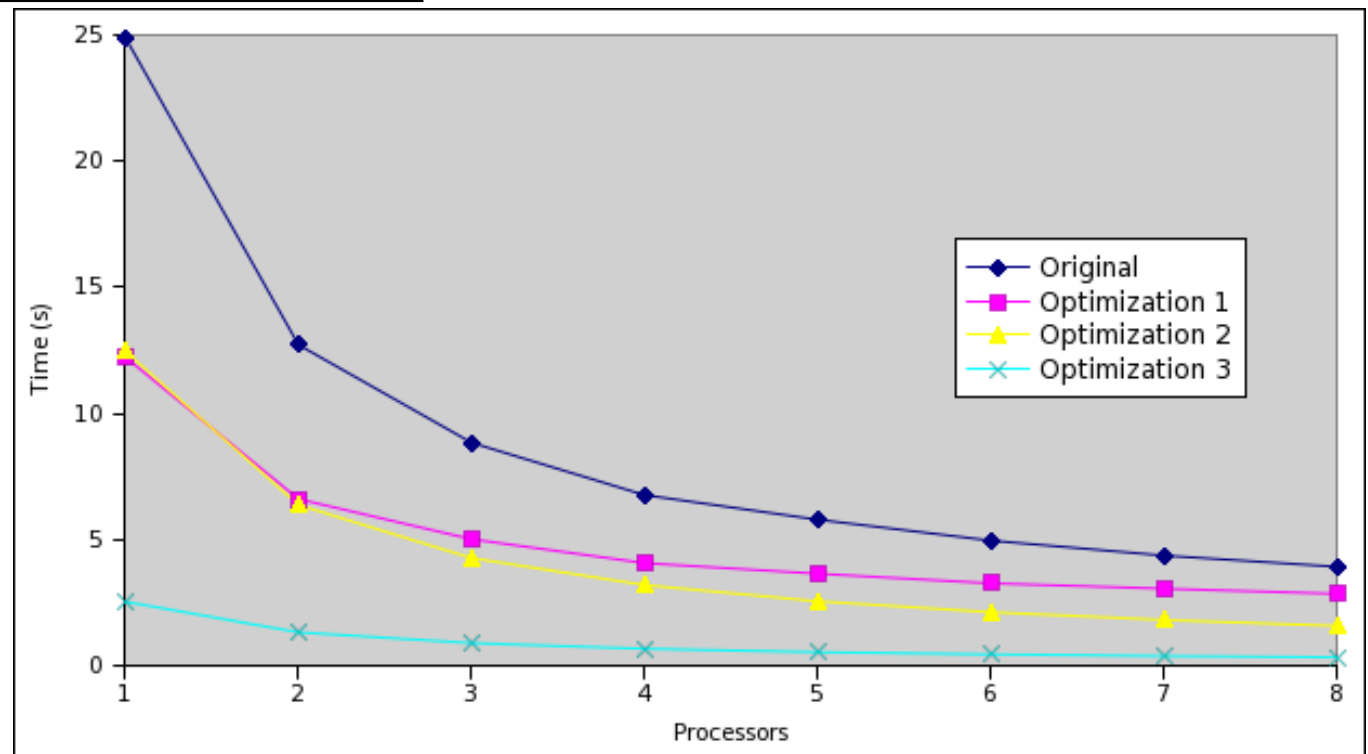
Improvements to the Program III

- Solution: reorganize loops
 - Mark each element of the array for all sieves between 3 and $\sqrt{n}$

3	5	7	9
11	13	15	17
19	21	23	25
27	29	31	33
35	37	39	41
43	45	47	49
51	53	55	57
59	61	63	65
67	69	71	73
75	77	79	81

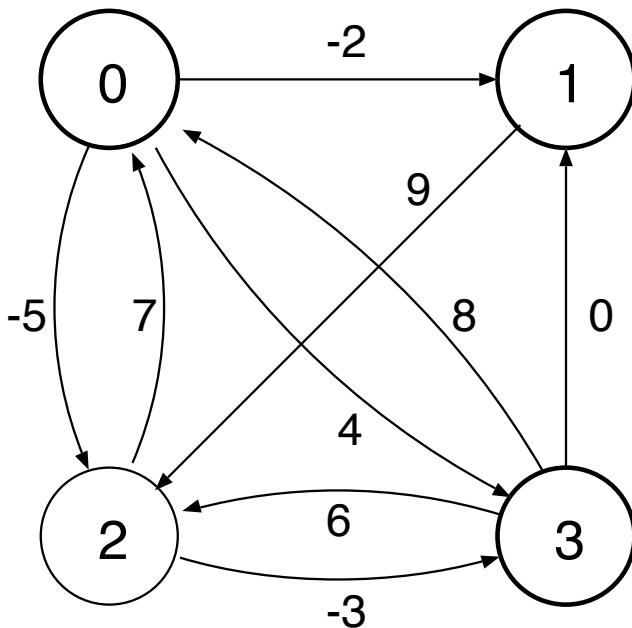
Improvements to the Program III

	Original	Opt. 1	Opt. 2	Opt. 3
1	24.9	12.2	12.5	2.5
2	13.0	6.6	6.4	1.3
3	9.0	5.0	4.3	0.9
4	7.1	4.1	3.2	0.7
5	6.9	3.7	2.6	0.5
6	5.2	3.3	2.1	0.5
7	4.7	3.1	1.8	0.4
8	4.2	2.9	1.6	0.3



All Pairs Shortest Paths

- Given a weighted, directed graph, determine the shortest path between any two vertices in the graph


$$\begin{bmatrix} 0 & -2 & -5 & 4 \\ \infty & 0 & 9 & \infty \\ 7 & \infty & 0 & -3 \\ 8 & 0 & 6 & 0 \end{bmatrix}$$

Adjacency Matrix

The Floyd-Warshall Algorithm

- Recursive solution based on **intermediate** vertices
- Let p_{ij} be the minimum-weight path from node i to node j among paths that use a subset of intermediate vertices $\{0, \dots, k-1\}$.
- Consider an additional node k :
 $k \notin p_{ij}$
 - then p_{ij} is shortest path in the subset of intermediate vertices $\{0, \dots, k\}$. $k \in p_{ij}$
 - then we can decompose p_{ij} as $i \xrightarrow{p_{ik}} k \xrightarrow{p_{kj}} j$, where subpaths p_{ik} and p_{kj} have intermediate vertices in the set $\{0, \dots, k-1\}$.

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k = -1 \\ \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) & \text{if } k \geq 0 \end{cases}$$

The Floyd-Warshall Algorithm

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k = -1 \\ \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) & \text{if } k \geq 0 \end{cases}$$

We do not consider self-loops, therefore:

for $k = 0$ **to** $|V|-1$

$$d_{ik}^{(k)} = d_{ik}^{(k-1)} \text{ and } d_{kj}^{(k)} = d_{kj}^{(k-1)}$$

for $i = 0$ **to** $|V|-1$

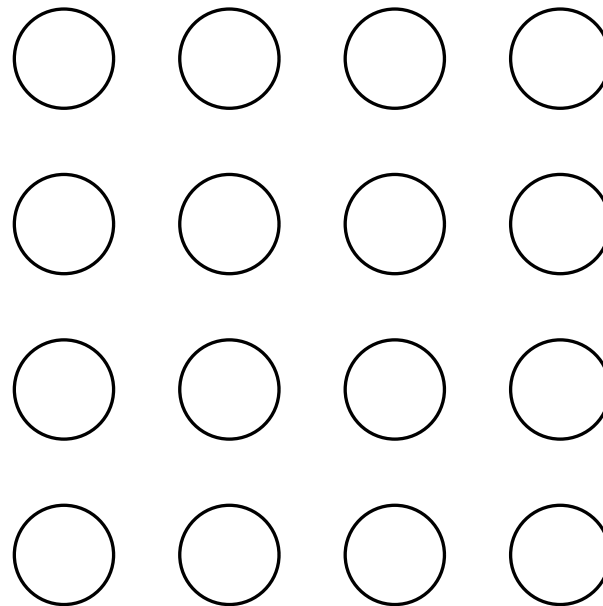
for $j = 0$ **to** $|V|-1$

$$d[i, j] = \min(d[i, j], d[i, k] + d[k, j])$$

Complexity: $\Theta(|V|^3)$

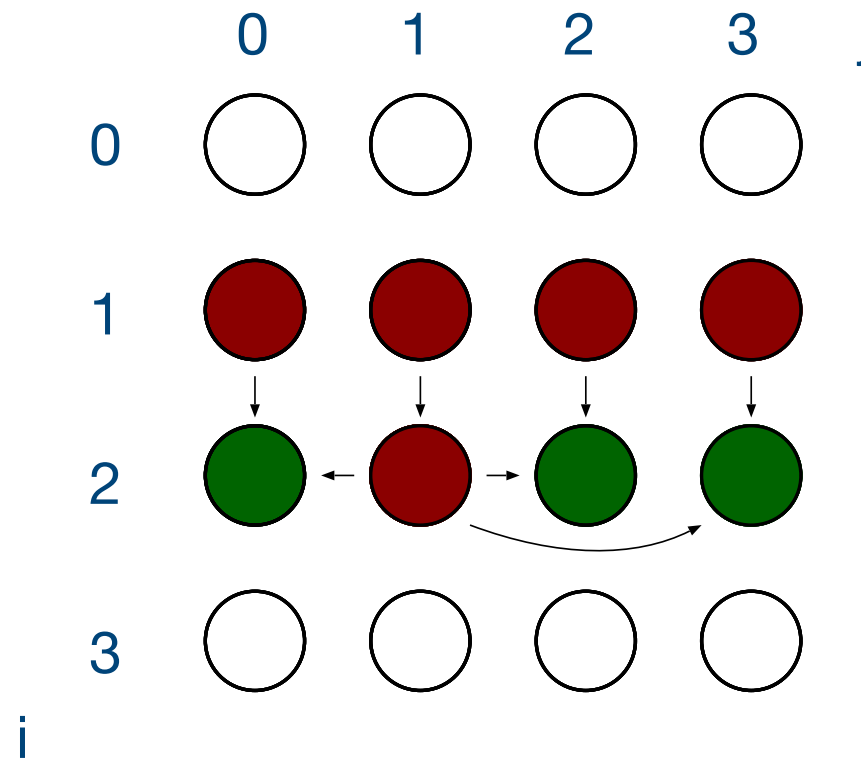
Foster's Methodology on Floyd-Warshall

- **Partitioning**
 - Domain decomposition: divide adjacency matrix into its $|V|^2$ elements (computation in the inner loop is primitive task)



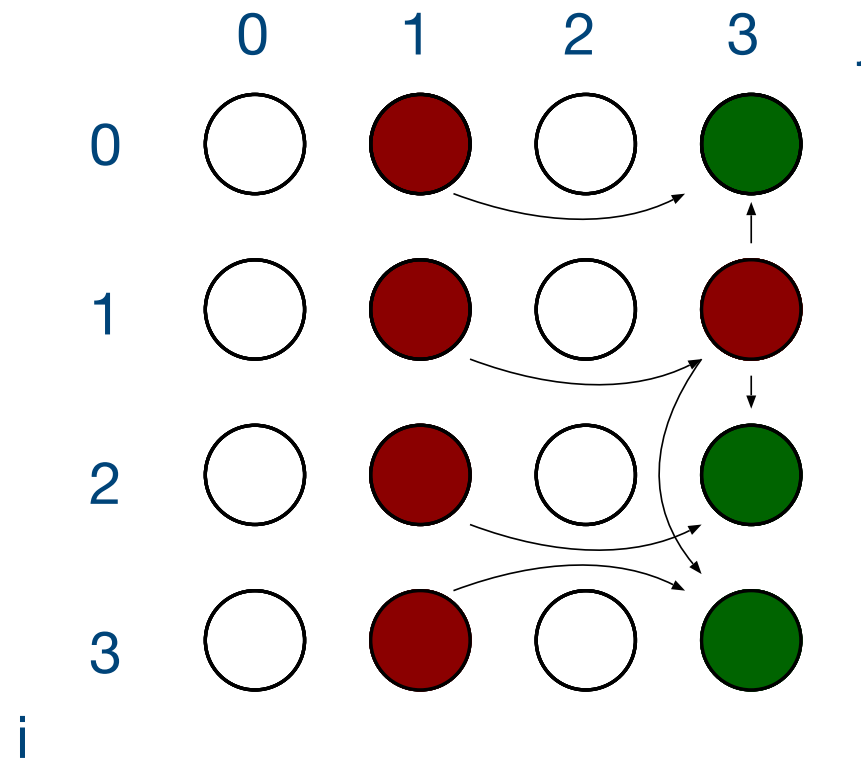
Foster's Methodology on Floyd-Warshall

- **Communication**
 - Let $k = 1$. Row sweep, $i = 2$.



Foster's Methodology on Floyd-Warshall

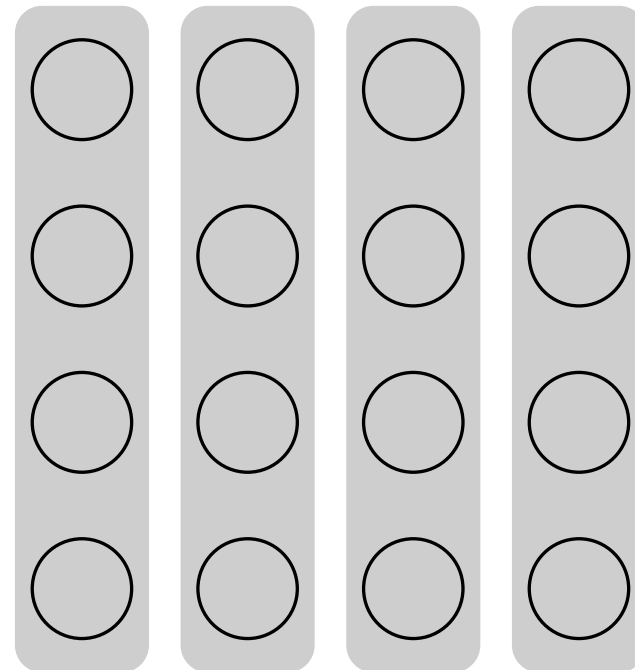
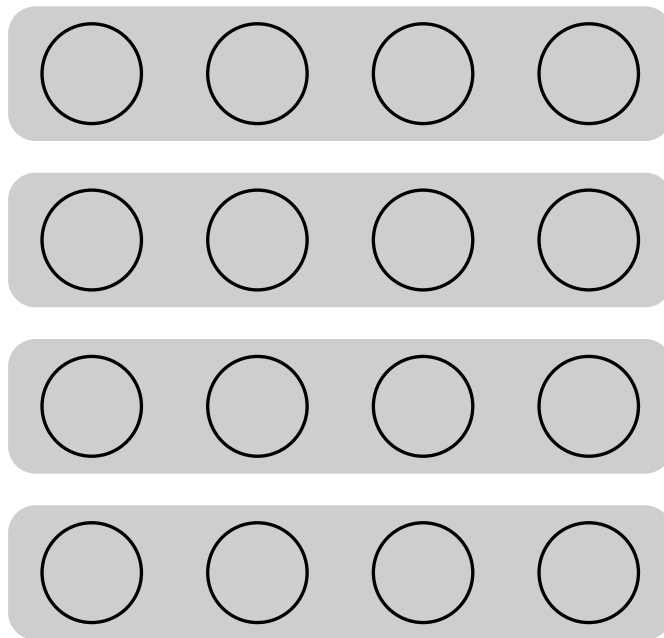
- **Communication**
 - Let $k = 1$. Column sweep, $j = 3$.



Foster's Methodology on Floyd-Warshall

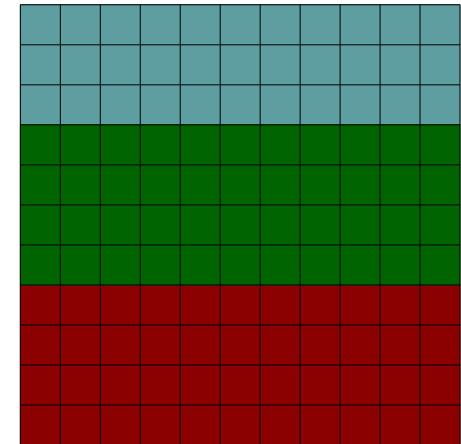
- **Communication**

- In iteration k , every task in row/column k broadcasts its value within task row/column

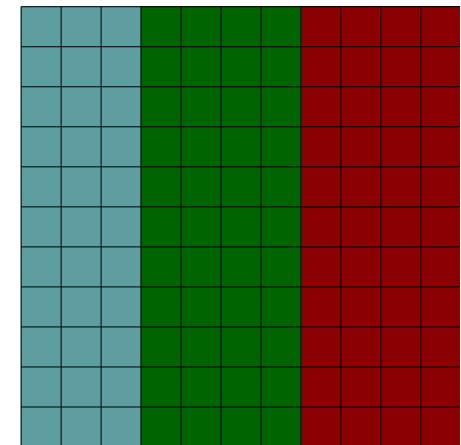


Foster's Methodology on Floyd-Warshall

- **Agglomeration and Mapping**
 - Agglomerate n^2 primitive tasks in p tasks, one per MPI process
 - Agglomerate tasks to minimize communication
 - Possible decompositions: block striped, **row-wise** or **column-wise** ($n=11$, $p=3$)
 - Relative merit?
 - Column-wise block striped
 - Broadcast within columns eliminated
 - Row-wise block striped
 - Broadcast within rows eliminated
 - Reading, writing and printing matrix simpler



row-wise

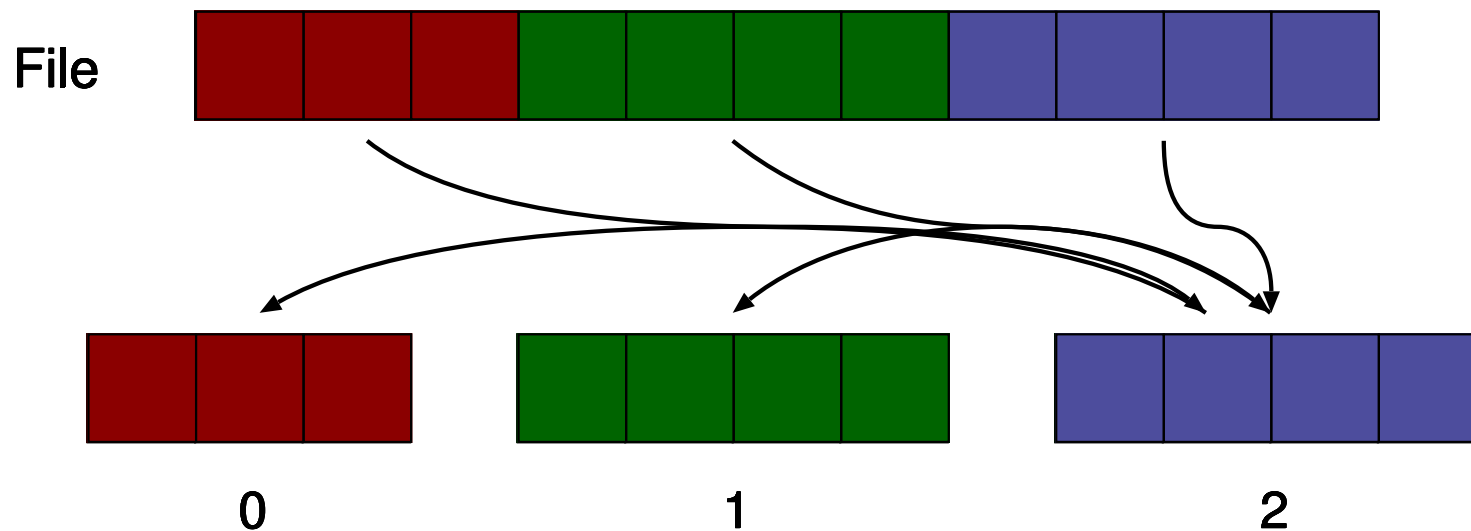


column-wise

Comparing Decompositions

- Choose row-wise block striped decomposition
- Some tasks get $\left\lceil \frac{n}{p} \right\rceil$ rows, others get $\left\lfloor \frac{n}{p} \right\rfloor$
- Which task gets which size?
- **Distributed approach:** distributed larger blocks evenly
 - First element of task : $\left\lfloor i \frac{n}{p} \right\rfloor$
 - Last element of task : $\left\lfloor (i + 1) \frac{n}{p} \right\rfloor - 1$
 - Task owner of element : $\left\lfloor \frac{p(j + 1) - 1}{n} \right\rfloor$

Reading the Graph Matrix



- Why don't we read the whole file and then execute a `MPI_Scatter`?

C Code

```
void compute_shortest_paths (int id, int p, double **a, int n)
{
    int i, j, k;
    int offset;    /* Local index of broadcast row */
    int root;      /* Process controlling row to be bcast */
    double* tmp;   /* Holds the broadcast row */

    tmp = (double *) malloc (n * sizeof(double));
    for (k = 0; k < n; k++) {
        root = BLOCK_OWNER(k,p,n);
        if (root == id) {
            offset = k - BLOCK_LOW(id,p,n);
            for (j = 0; j < n; j++)
                tmp[j] = a[offset][j];
        }
        MPI_Bcast (tmp, n, MPI_DOUBLE, root, MPI_COMM_WORLD);
        for (i = 0; i < BLOCK_SIZE(id,p,n); i++)
            for (j = 0; j < n; j++)
                a[i][j] = MIN(a[i][j], a[i][k] + tmp[j]);
    }
    free (tmp);
}
```

Analysis of the Parallel Algorithm

- Let α be the time to compute an iteration
- Sequential execution time: αn^3
- Computation time of parallel program: $\alpha n \left\lceil \frac{n}{p} \right\rceil n$
 - Innermost loop executed n times
 - Middle loop executed at most $\left\lceil \frac{n}{p} \right\rceil$ times
 - Outer loop executed n times
- Number of broadcasts: n
 - One per outer loop iteration
- Broadcast time: $\lceil \log p \rceil \left(\lambda + \frac{8n}{\beta} \right)$
 - Each broadcast has $\lceil \log p \rceil$ steps
 - λ is the message latency
 - β is the bandwidth
 - Each broadcast sends $8n$ bytes
- Expected parallel execution time $\alpha n^2 \left\lceil \frac{n}{p} \right\rceil + n \lceil \log p \rceil \left(\lambda + \frac{8n}{\beta} \right)$

Analysis of Parallel Algorithm

- Previous expression will overestimate parallel execution time
 - After the first iteration, broadcast transmission time overlaps with computation of next row

- Expected parallel execution time

$$\alpha n^2 \left\lceil \frac{n}{p} \right\rceil + n \lceil \log p \rceil \lambda + \lceil \log p \rceil \frac{8n}{\beta}$$

- Experimental measurements

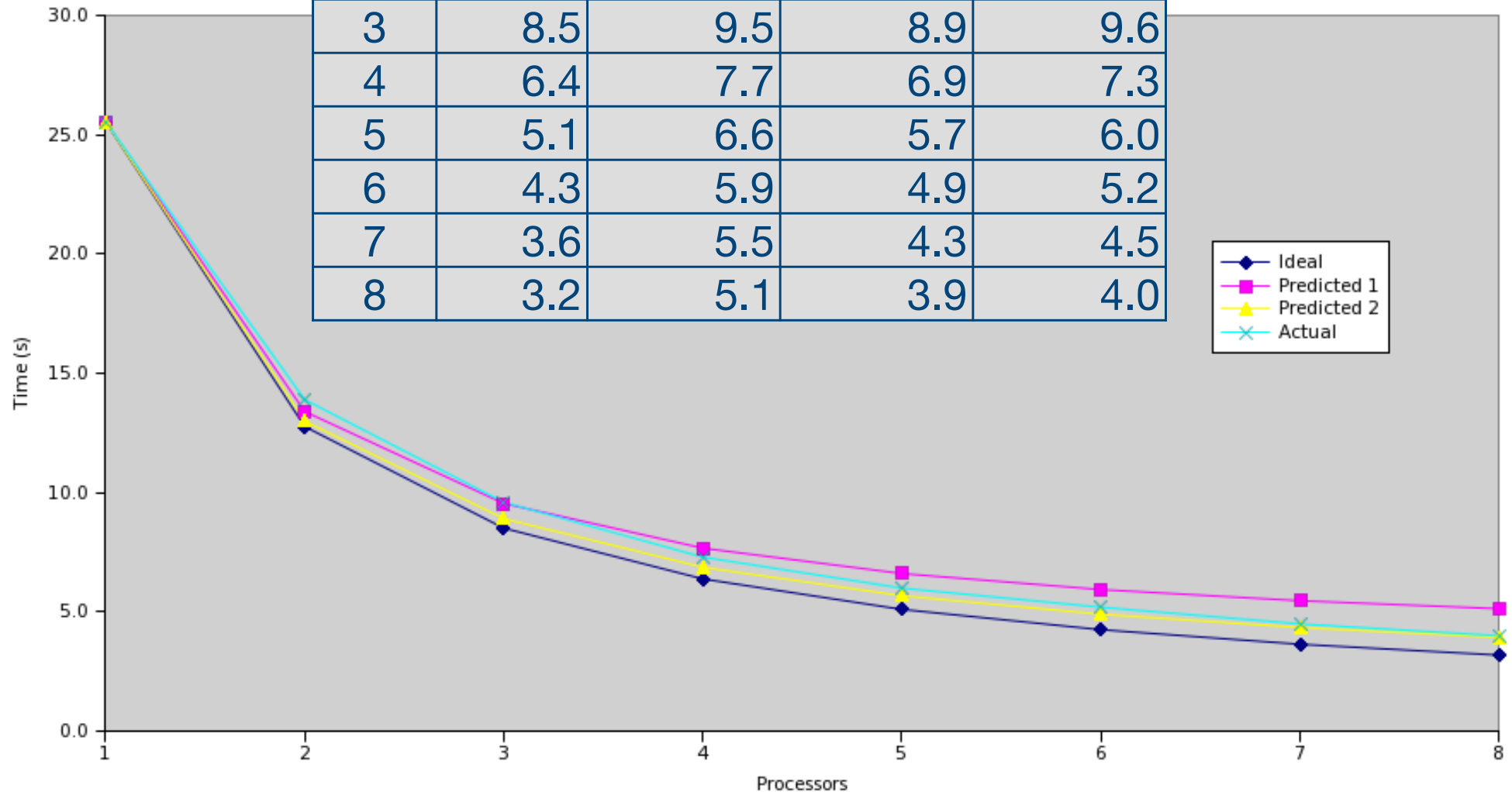
$$\alpha = 25,5 \text{ ns}$$

$$\lambda = 250 \mu\text{s}$$

$$\beta = 10^7 \text{ bytes/s}$$

Experimental Results

Procs	Ideal	Predict 1	Predict 2	Actual
1	25.5	25.5	25.5	25.5
2	12.8	13.4	13.0	13.9
3	8.5	9.5	8.9	9.6
4	6.4	7.7	6.9	7.3
5	5.1	6.6	5.7	6.0
6	4.3	5.9	4.9	5.2
7	3.6	5.5	4.3	4.5
8	3.2	5.1	3.9	4.0



Review

- Application examples with MPI
 - The Sieve of Eratosthenes
 - All-Pairs Shortest Paths, Floyd-Warshall's Algorithm

Next Class ...

- Performance Analysis
 - Speedup and Efficiency
 - Performance Prediction Formulas
 - Amdahl's Law
 - Gustafson-Barsis' Law
 - Karp-Flatt Metric
 - Isoefficiency Metric
- Matrix-vector multiplication
 - Decompositions
 - Row-wise
 - Column-wise
 - Checkerboard
 - Gather, scatter, alltoall
 - Grid-oriented communications